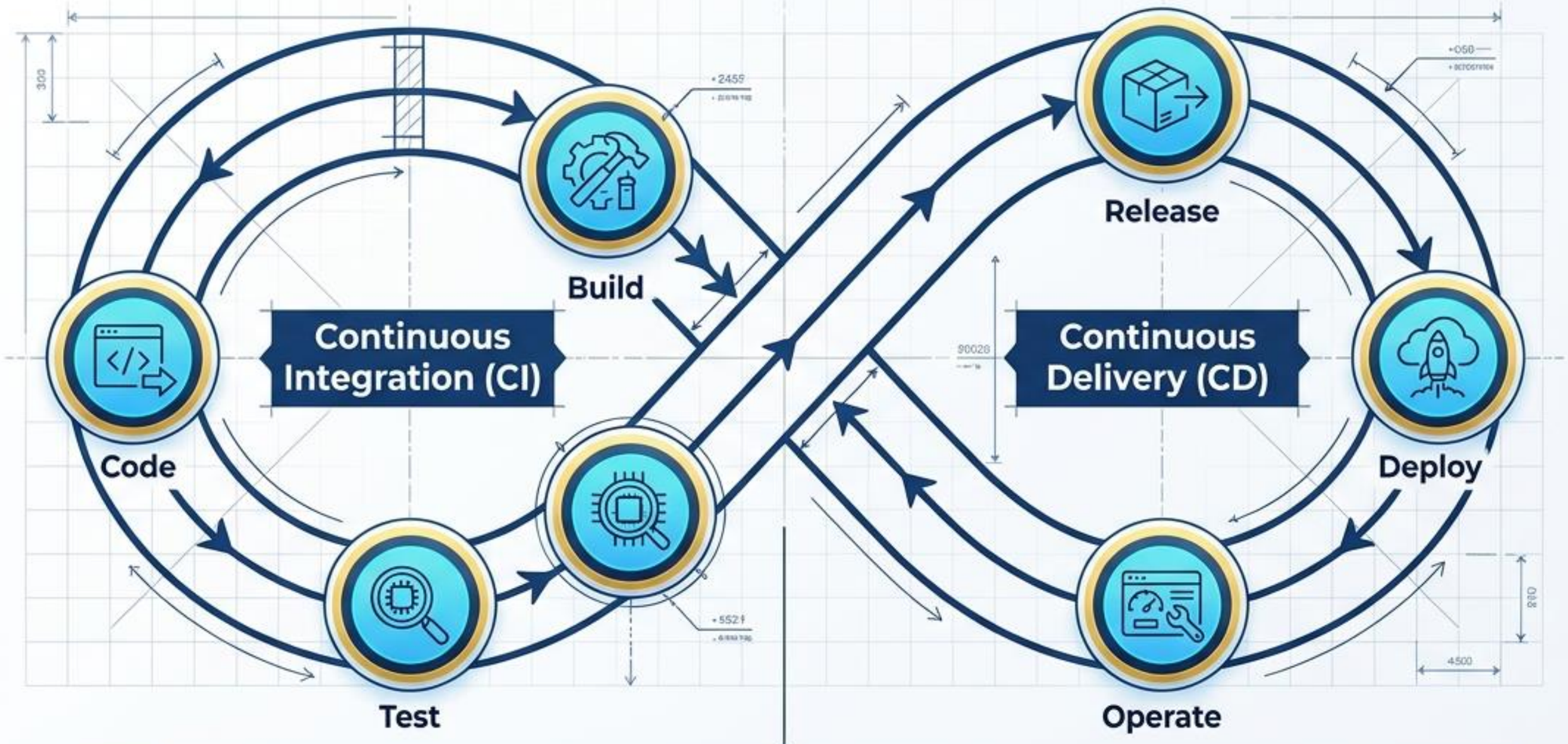
Ulster University

Creating a CI/CD Pipeline on Azure

The Blueprint for Continuous Integration & Deployment

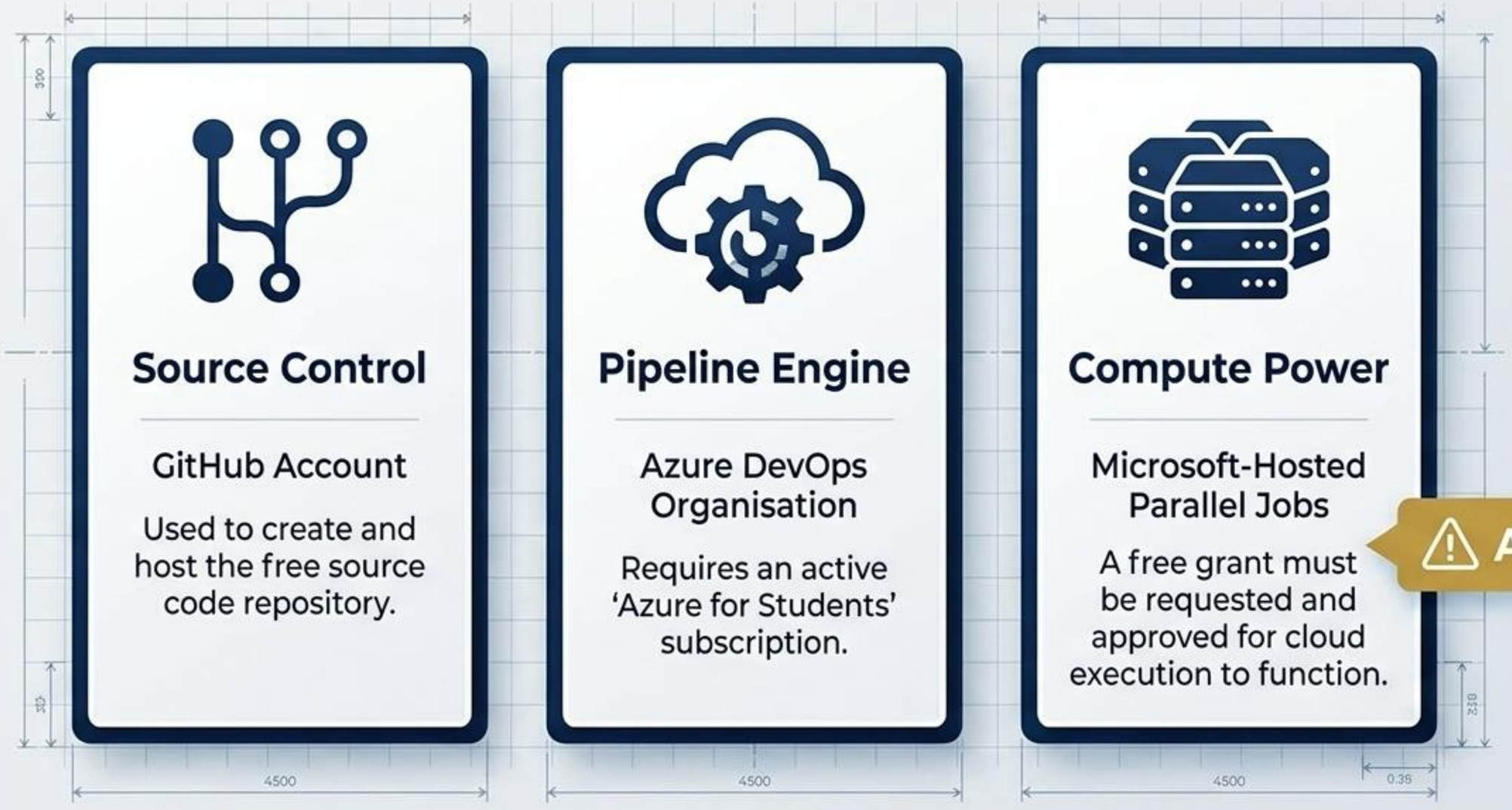
Week 5 Practical | Azure DevOps & GitHub Integration

Decoding the CI/CD Engine



Azure Pipelines automates this entire lifecycle. It transforms manual code updates on GitHub into automated, cloud-hosted build-deploy-test workflows, enabling fast and scalable software routines.

Project Prerequisites: The Required Arsenal



Source Control

GitHub Account

Used to create and host the free source code repository.

Pipeline Engine

Azure DevOps Organisation

Requires an active ‘Azure for Students’ subscription.

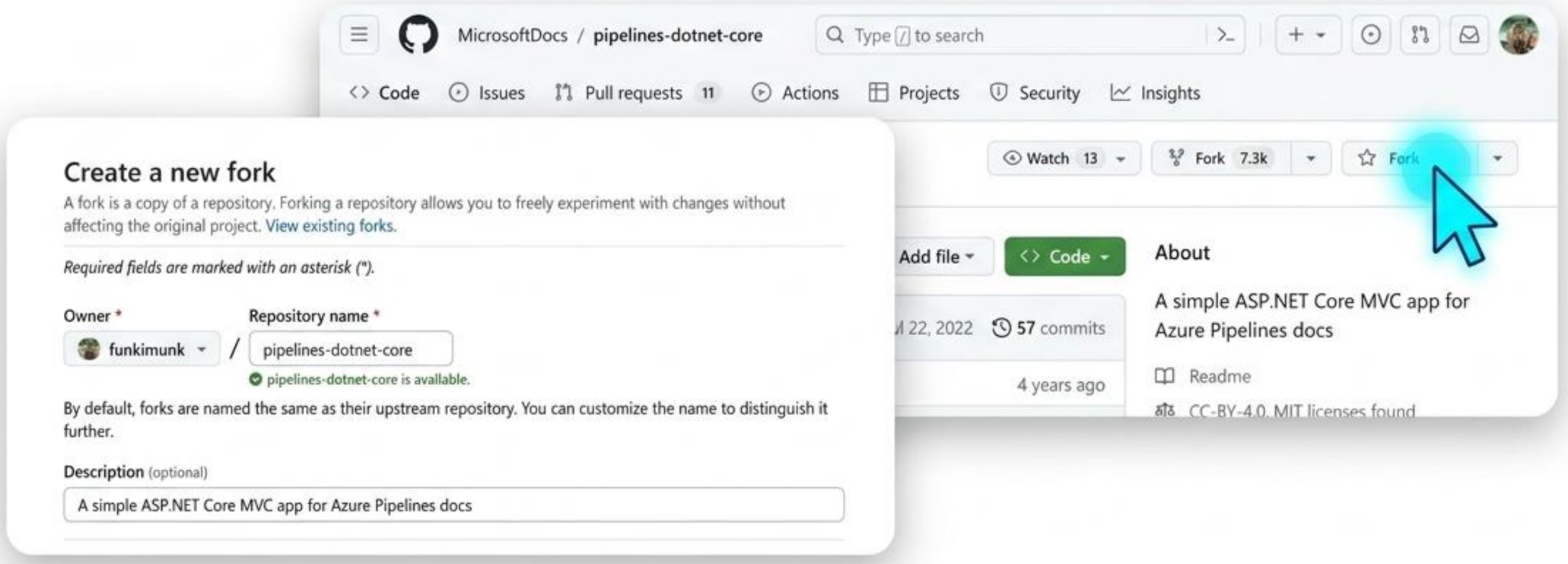
Compute Power

Microsoft-Hosted Parallel Jobs

A free grant must be requested and approved for cloud execution to function.

Alert

Phase 1: Forking the Source Control

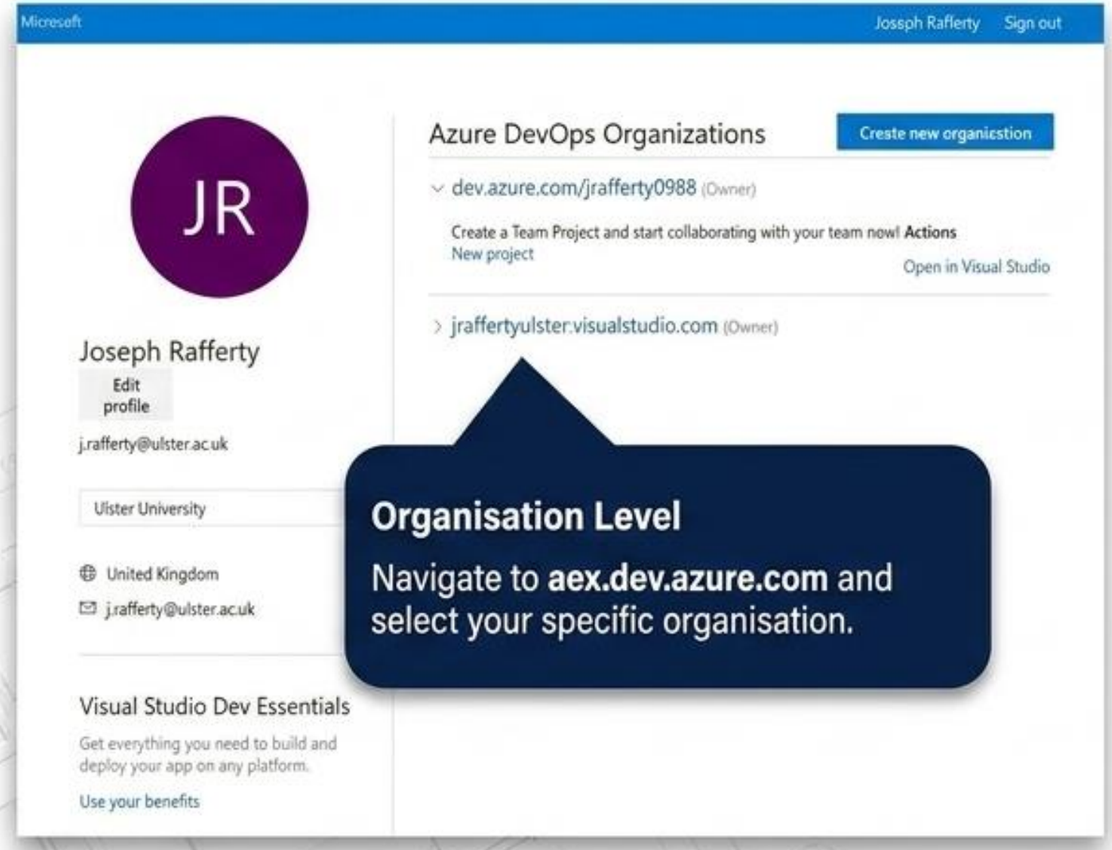


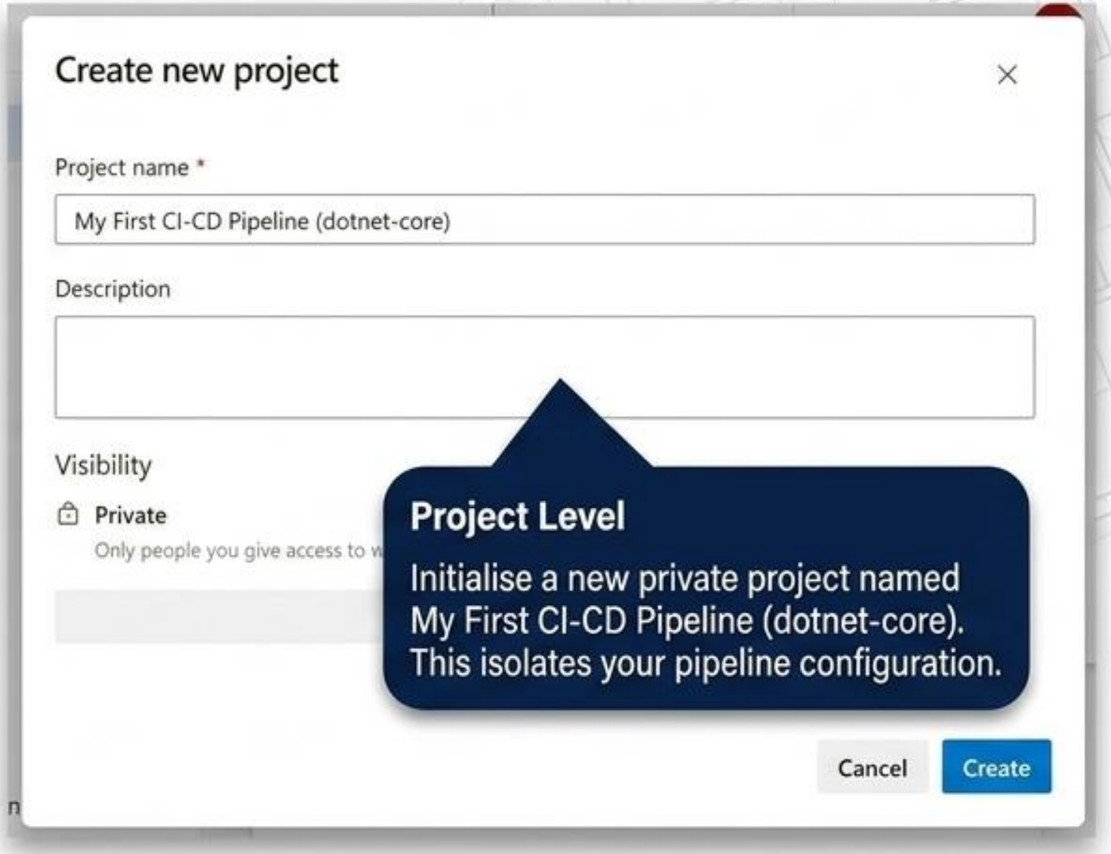
1. Navigate to the target repository (**MicrosoftDocs/pipelines-dotnet-core**).

2. Click **Fork** to clone the remote codebase directly into your personal GitHub account.

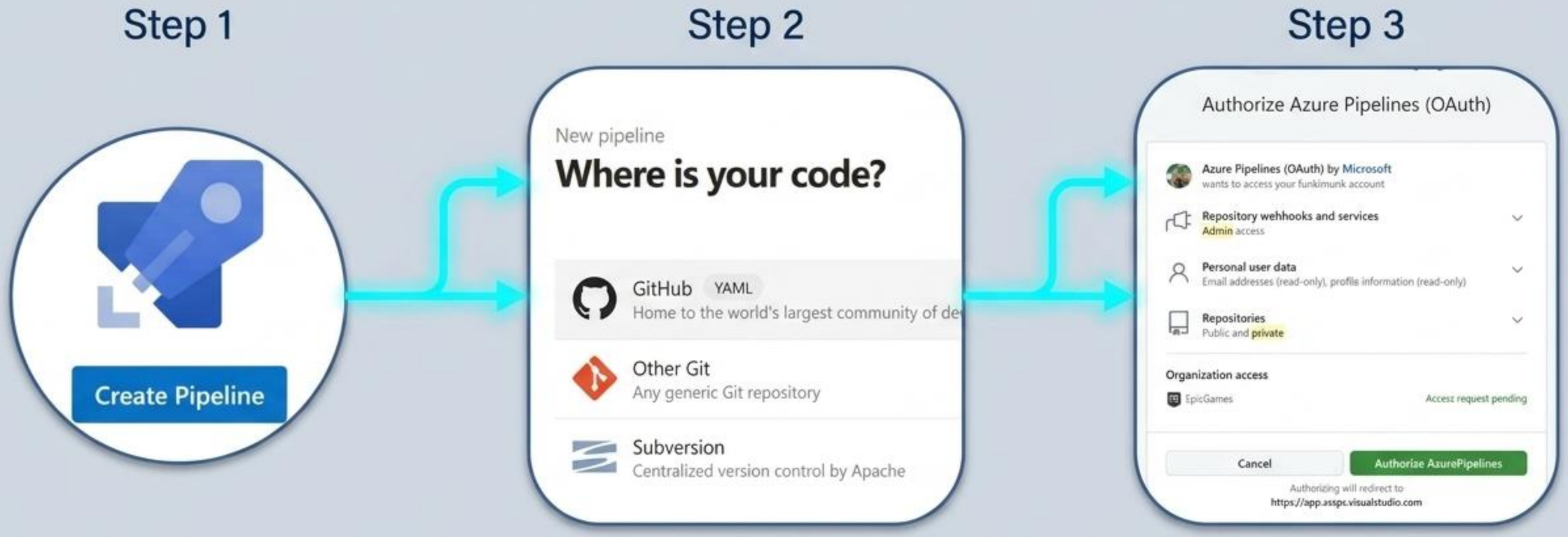
3. Confirm yourself as the **Owner** and retain the default repository name to prevent conflicts.

Phase 2: Initialising the DevOps Sandbox



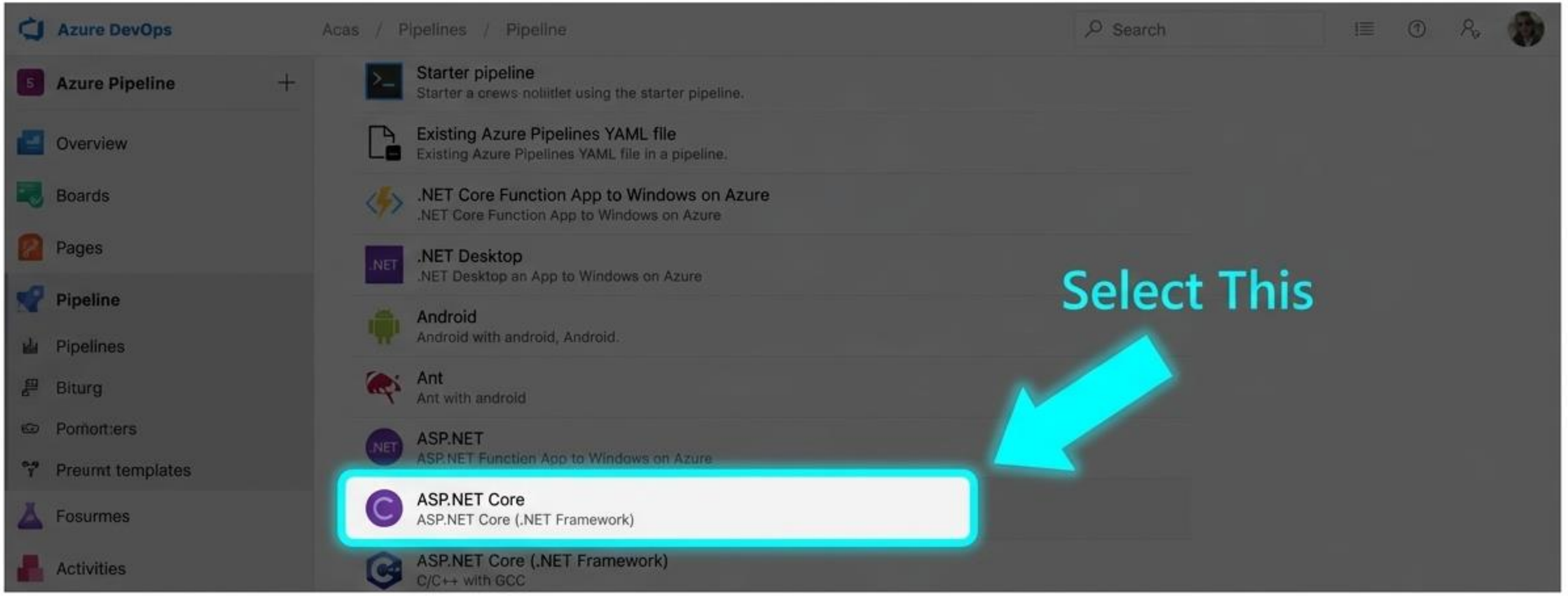


Phase 3: Building the Integration Bridge (OAuth)



Azure DevOps requires explicit read/write access to your GitHub repository metadata. Approving this OAuth handshake allows Azure to detect code commits automatically and trigger the build pipeline without manual intervention.

Phase 4: Selecting the Pipeline Blueprint



Note:

The forked GitHub repository contains an ASP.NET Core MVC application. You must select the matching ASP.NET Core YAML template to ensure Azure provisions the correct .NET build environment for compilation.

Infrastructure as Code: Dissecting the YAML

```yaml
# ASP.NET Core
# Build and test ASP.NET Core projects targeting .NET Core.
# Add steps that run tests, create a NuGet package, deploy, and more:
# https://docs.microsoft.com/azure/devops/pipelines/languages/dotnet-core

trigger:
- master

pool:
  vmImage: ubuntu-latest

variables:
  buildConfiguration: 'Release'

steps:
- script: dotnet build --configuration $(buildConfiguration)
  displayName: 'dotnet build $(buildConfiguration)'

```

The Event

Instructs Azure to run the pipeline automatically anytime new code is pushed to this branch.

The Environment

Provisions a fresh Ubuntu Linux virtual machine in the cloud to act as the build server.

The Action

The specific terminal command the agent executes to compile the application.

Phase 5: Execution & The Parallelism Roadblock

The Problem

New Azure DevOps accounts restrict Microsoft-hosted agents to prevent abuse.

If your pipeline fails with a “No hosted parallelism” error, the build cannot start.

We've received your request to increase free parallelism in Azure DevOps.

Please note that your request was Completed

Request Details:

Name	Joe Rafferty
Email	j.rafferty@ulster.ac.uk
Organization Name	jrafferty0988
Parallelism Type	Private

The Resolution

A free parallelism grant must be requested via Microsoft.

Once the confirmation email arrives, the stalled pipeline can be rerun successfully.

Compute Alternatives: Cloud vs. Local Agents

	Microsoft-Hosted Agent (Cloud)	Local Agent (Self-Hosted)
Maintenance	Zero (Microsoft handles VM updates)	High (User manages software environment)
Access / Cost	Requires approved grant and wait time	Immediate access, bypasses grant delays
Performance	Standardised, predictable VM	Dependent on the hardware specifications of your PC

Takeaway: Establishing a local Windows agent is the recommended immediate workaround if grant approval is delayed.

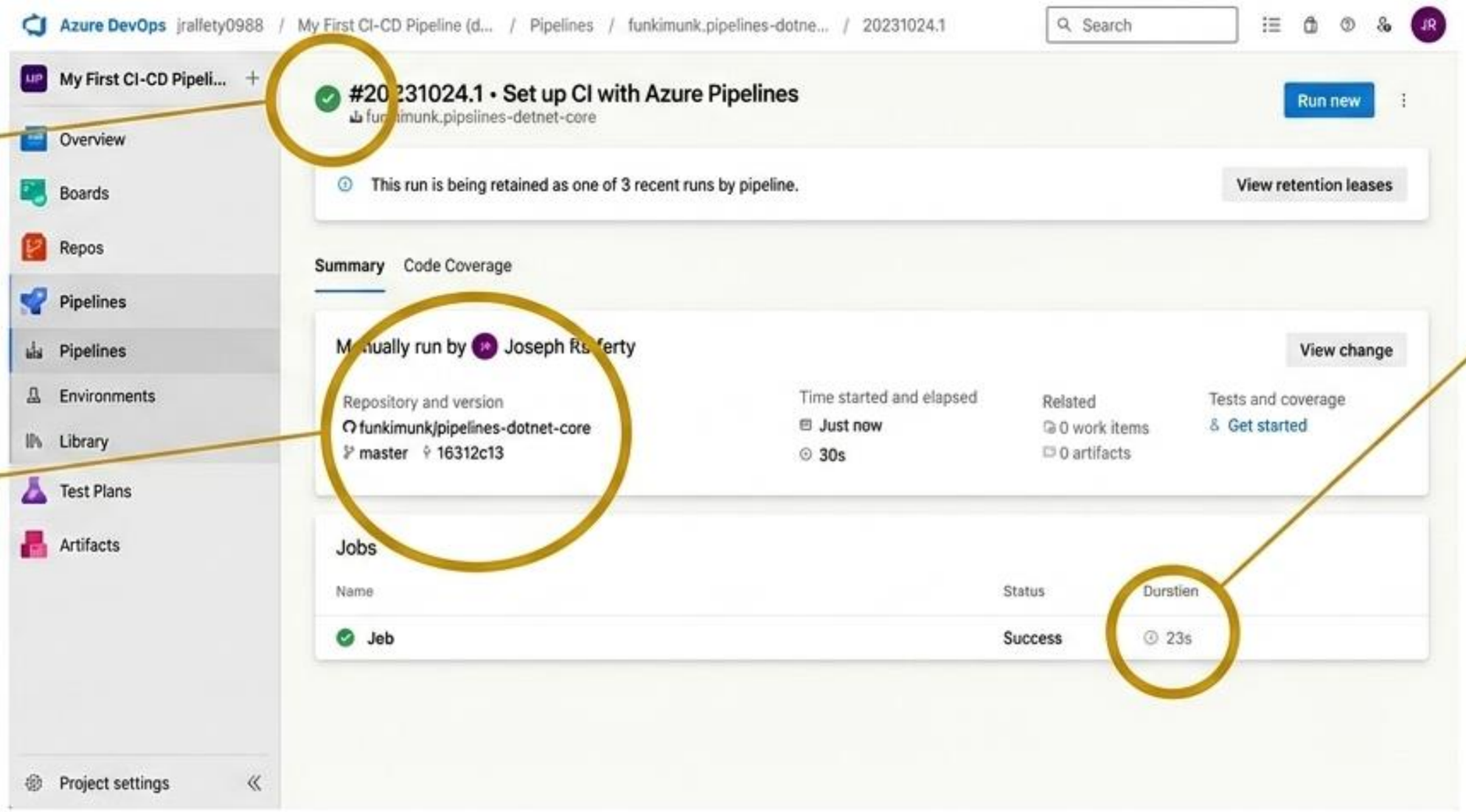
Validating Pipeline Success

Status

The green checkmark confirms a successful compilation.

Trigger Metadata

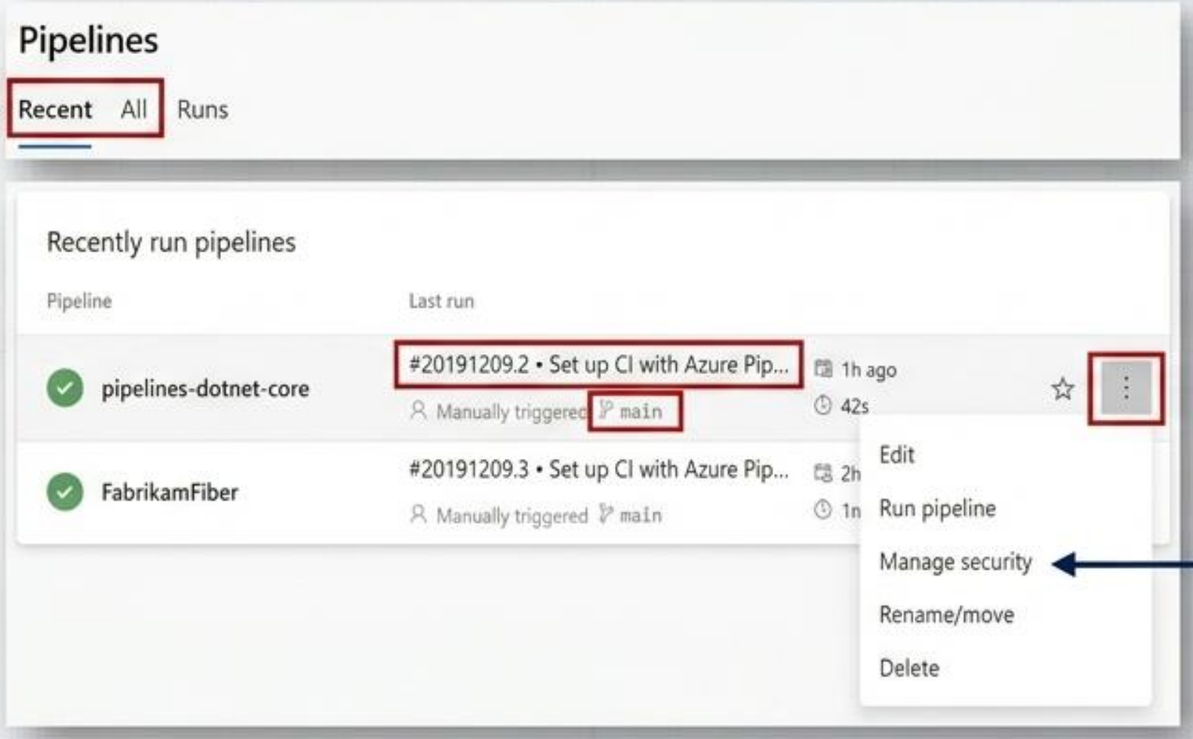
Shows the exact commit message and the target branch.



Performance

Displays the total duration metrics taken to provision the VM and build the code.

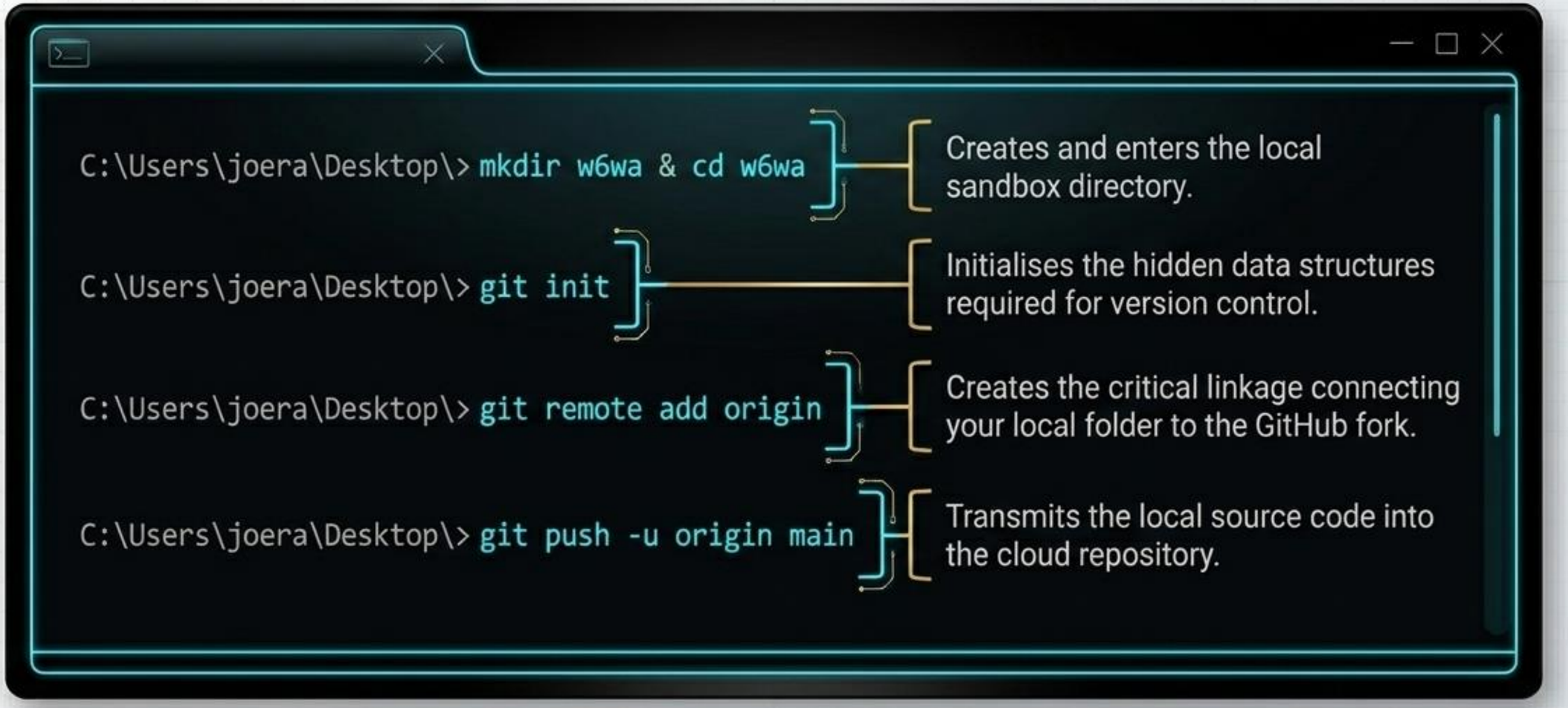
Managing Pipeline Health & History



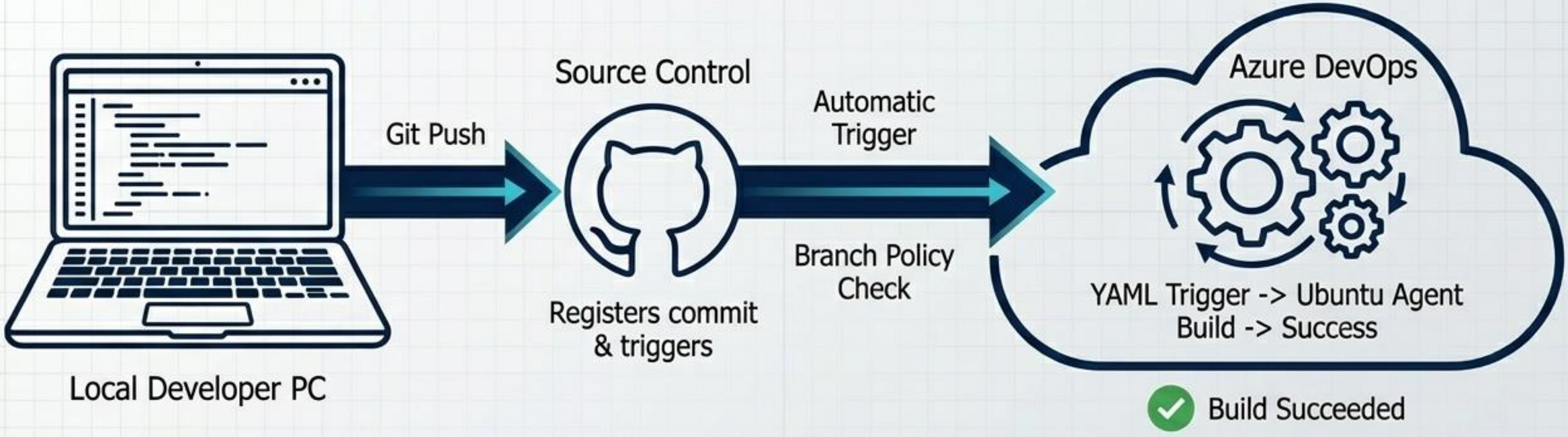
Dashboard Navigation
Use the top tabs to toggle between recently run pipelines and the complete historical log of all individual runs.

Run Management
The context menu (...) allows you to manually re-run pipelines, edit the YAML directly, or apply retention policies to lock specific historical builds from being auto-deleted.

Local to Cloud: Initialising Git via Terminal



The Complete Code-to-Cloud Workflow



The Big Picture: You have successfully built a living, automated engine. A single command executed in a local terminal now safely builds, tests, and validates your software in the cloud without further manual intervention.

Resource Management & Next Steps

Resource Clean Up

Always delete Azure App Services and associated resources from the DevOps Starter dashboard once testing is complete to prevent unwanted charges to your student account.

Further Learning

With Continuous Integration (CI) now successfully established, the infrastructure is ready for the next evolution: configuring deployment stages for Continuous Delivery (CD).